

Projet d'Optimisation - Finance

Recherche d'un cycle maximisant

Javier Peña Castano, Martin Leiva, Baptiste Pras

Université Paris-Saclay

Lundi 20 Octobre 2025

Problème

	EURO	USD	JP	CHF
EURO	1	1.19	1.33	1.62
USD	0.84	1	1.12	1.37
JP	0.75	0.89	1	1.22
CHF	0.62	0.73	0.82	1

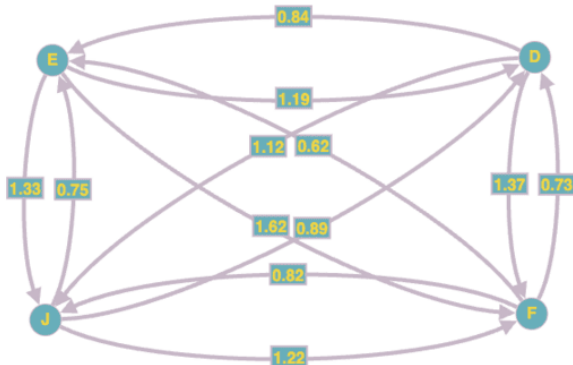
Taux de change entre différentes monnaies



Modélisation du problème

- Graphe orienté $G = (X, U)$ avec $X = \{E, D, F, J\}$
- Chaque arête (x_i, x_j) représente le taux de change x_i à x_j
- Objectif : trouver un cycle de n sommets partant de E qui maximise

$$v(\mu) = \prod_{i=0}^{n-1} v(x_i, x_{i+1})$$



Structure du graphe en Python

Listing 1 – Dictionnaire d'adjacence de G

```
G = {  
    'E': {'D': 1.19, 'J': 1.33, 'F': 1.62},  
    'D': {'E': 0.84, 'J': 1.12, 'F': 1.37},  
    'J': {'E': 0.75, 'D': 0.89, 'F': 1.22},  
    'F': {'E': 0.62, 'D': 0.73, 'J': 0.82}  
}
```

Algorithme 1 — Recherche exhaustive (DFS)

Algorithm 1 `meilleur_cycle( $G$ ,  $debut$ ,  $p$ )` - Temps: $O(|X|^p)$ - Mémoire: $O(p)$

```
1: Entrées : dictionnaire d'adjacence  $G$  des taux  $G[i, j]$ , monnaie de départ  $debut$ , nombre max  
   d'échanges  $p$   
2: Sorties : ( $meilleur\_gain$ ,  $meilleur\_chemin$ ) le cycle partant de  $debut$  avec  $\leq p$  échanges max-  
   imisant les gains  
3: procédure DFS( $noeud$ ,  $gain$ ,  $chemin$ ,  $profondeur$ )  
4:    $meilleur\_gain \leftarrow 0.0$   
5:    $meilleur\_chemin \leftarrow []$   
6:   if  $noeud = debut$  and  $profondeur > 0$  then  
7:      $meilleur\_gain \leftarrow gain$   
8:      $meilleur\_chemin \leftarrow chemin$   
9:   if  $profondeur = p$  then  
10:    return ( $meilleur\_gain$ ,  $meilleur\_chemin$ )  
11:  for each  $sommet$ ,  $arete$  in  $G[noeud]$  do  
12:    ( $gain'$ ,  $chemin'$ )  $\leftarrow$  DFS( $sommet$ ,  $gain \times arete$ ,  $chemin \cup [sommet]$ ,  $profondeur + 1$ )  
13:    if  $gain' > meilleur\_gain$  then  
14:       $meilleur\_gain \leftarrow gain'$   
15:       $meilleur\_chemin \leftarrow chemin'$   
16: return DFS( $debut$ , 1.0, [ $debut$ ], 0)
```

Algorithme 2 — Programmation dynamique (type Bellman-Ford)

Algorithme 2 *meilleur_cycle.ameliore*(G , $debut$, p) - Temps: $O(p|U|)$ - Mémoire: $O(p|X|)$

```
1: Entrées : dictionnaire d'adjacence  $G$  des taux  $G[i, j]$ , monnaie de départ  $debut$ , nombre max  
   d'échanges  $p$   
2: Sorties : ( $meilleur\_gain$ ,  $meilleur\_chemin$ ) le cycle partant de  $debut$  avec  $\leq p$  échanges max-  
   imisant les gains  
3:  $dp[k][sommet] \leftarrow 0.0$  for all  $k \in [0, p]$  et  $sommet \in G$   
4:  $parent[k][sommet] \leftarrow \text{NULL}$  for all  $k \in [0, p]$  et  $sommet \in G$   
5:  $dp[0][debut] \leftarrow 1.0$   
6: for  $k \leftarrow 1$  to  $p$  do  
7:    $flag \leftarrow \text{False}$   
8:   for each  $sommet$  in  $G$  do  
9:     if  $dp[k-1][sommet] \leq 0.0$  then  
10:      Continue  
11:    $gain \leftarrow dp[k-1][sommet]$   
12:   for each  $sommet', arete$  in  $G[sommet]$  do  
13:      $gain' \leftarrow gain \times arete$   
14:     if  $gain' > dp[k][sommet']$  then  
15:        $dp[k][sommet'] \leftarrow gain'$   
16:        $parent[k][sommet'] \leftarrow sommet$   
17:    $flag \leftarrow \text{True}$ 
```

```
18:   if not  $flag$  then  
19:     break  
20:    $meilleur\_gain \leftarrow 0.0$   
21:    $meilleur\_k \leftarrow \text{NULL}$   
22:   for  $k \leftarrow 1$  to  $p$  do  
23:     if  $dp[k][debut] > meilleur\_gain$  then  
24:        $meilleur\_gain \leftarrow dp[k][debut]$   
25:        $meilleur\_k \leftarrow k$   
26:   if  $meilleur\_k$  is  $\text{NULL}$  then  
27:     return  $0.0 \quad []$   
28:    $cycle \leftarrow []$   
29:    $monnaie \leftarrow debut$   
30:   for  $i \leftarrow meilleur\_k$  to  $1$  with step  $-1$  do  
31:      $tmp \leftarrow parent[i][monnaie]$   
32:      $cycle.append(tmp)$   
33:      $monnaie \leftarrow tmp$   
34:    $cycle.reverse()$   
35:    $meilleur\_cycle \leftarrow cycle \cup [debut]$   
36:   return ( $meilleur\_gain$ ,  $meilleur\_cycle$ )
```

Algorithme 3 — Programmation dynamique & unbounded knapsack

Algorithme 3 `meilleur_cycle.optimal( $G$ ,  $debut$ ,  $p$ )` - Temps: $O(|X||U| + p|X|)$ - Mémoire: $O(|X|^2 + p)$

```
1: Entrées : dictionnaire d'adjacence  $G$  des taux  $G[t, j]$ , monnaie de départ  $debut$ , nombre max  
d'échanges  $p$   
2: Sorties :  $\langle meilleur\_gain, meilleur\_chemin \rangle$  le cycle partant de  $debut$  avec  $\leq p$  échanges max-  
imisant les gains  
3:  $X \leftarrow |G(X)|$   
4:  $dp[k][sommet] \leftarrow 0.0$  for all  $k \in [0, p]$  et  $sommet \in G$   
5:  $parent[k][sommet] \leftarrow \text{NULL}$  for all  $k \in [0, p]$  et  $sommet \in G$   
6:  $dp[0][debut] \leftarrow 1.0$   
7: for  $k \leftarrow 1$  to  $p$  do  
8:    $flag \leftarrow \text{False}$   
9:   for each  $sommet$  in  $G$  do  
10:    if  $dp[k-1][sommet] < 0.0$  then  
11:      Continue  
12:     $gain \leftarrow dp[k-1][sommet]$   
13:    for each  $sommet', arête$  in  $G[sommet]$  do  
14:       $gain' \leftarrow gain \times arête$   
15:      if  $gain' > dp[k][sommet']$  then  
16:         $dp[k][sommet'] \leftarrow gain'$   
17:         $parent[k][sommet'] \leftarrow sommet$   
18:         $flag \leftarrow \text{True}$   
19:    if not flag then  
20:      break  
21: If  $dp[k][debut] > 0.0$  then  
22:    $cycle\_inverse \leftarrow []$   
23:    $monnaie \leftarrow debut$   
24:   for  $i \leftarrow meilleur\_k$  to  $1$  with step  $-1$  do  
25:      $tmp \leftarrow parent[i][monnaie]$   
26:      $cycle.append(tmp)$   
27:      $monnaie \leftarrow tmp$   
28:    $cycle.reverse()$   
29:    $meilleur\_cycle\_simple[k] \leftarrow (dp[k][debut], cycle\_inverse \cup [debut])$ 
```

```
31:  $parent2[t] \leftarrow \text{NULL}$  for all  $t \in [0, p]$   
32:  $dp2[0] \leftarrow 1.0$   
33: for  $t \leftarrow 1$  à  $p$  do  
34:   for each  $(k, (gain, cycle))$  in  $meilleur\_cycle\_simple$  do  
35:     if  $k \leq t$  et  $dp2[t-k] > 0.0$  then  
36:        $nouveau \leftarrow dp2[t-k] \times gain$   
37:       if  $nouveau > dp2[t]$  then  
38:          $dp2[t] \leftarrow nouveau$   
39:          $parent2[t] \leftarrow k$   
40:    $t' \leftarrow \arg \max_{t \in [0, p]} dp2[t]$   
41:    $meilleur\_gain \leftarrow dp2[t']$   
42:    $combo \leftarrow []$   
43:    $t \leftarrow t'$   
44:   while  $t > 0$  et  $parent2[t] \neq \text{NULL}$  do  
45:      $k \leftarrow parent2[t]$   
46:      $combo.append(meilleur\_cycle\_simple[k][1])$   
47:      $t \leftarrow t - k$   
48:    $combo.reverse()$   
49:    $fusion \leftarrow []$   
50:    $budget \leftarrow t'$   
51:   for each cycle in  $combo$  do  
52:     if  $budget = 0$  then  
53:       break  
54:     if fusion vide then  
55:        $take \leftarrow \min(budget, |cycle| - 1)$   
56:        $fusion \leftarrow cycle[1 : take + 1]$   
57:     else  
58:        $take \leftarrow \min(budget, |cycle| - 1)$   
59:        $fusion \leftarrow fusion + cycle[1 : 1 + take]$   
60:      $budget \leftarrow budget - take$   
61:   return  $(meilleur\_gain, fusion)$ 
```

Algorithme 4 — Le meilleur des deux mondes

$$|X| \geq 2 \quad (\text{sinon aucun échange possible})$$

$$|U| = |X|(|X| - 1) \quad (\text{nombre d'arêtes dans un graphe complet})$$

Quand l'algo 2 est-il plus lent que l'algo 3 ? Calculons !

$$p|U| \geq p(|X| - 1) + |X||U|$$

$$p|X|(|X| - 1) \geq p(|X| - 1) + |X|^2(|X| - 1)$$

$$\iff p|X| \geq p + |X|^2$$

$$\iff p|X| - p \geq |X|^2$$

$$\iff p(|X| - 1) \geq |X|^2$$

$$\iff \boxed{p \geq \frac{|X|^2}{|X| - 1}}.$$

$$\frac{|X|^2}{|X| - 1} = |X| + 1 + \frac{1}{|X| - 1} \quad \Rightarrow \quad \lim_{|X| \rightarrow +\infty} \frac{|X|^2}{|X| - 1} \xrightarrow{> |X| + 1} |X| + 1$$